Python in Research, Part 6

Discrete Event Simulation

Many problems in real life are very complex. One can construct a mathematical model to understand them. Yet, in some cases, solving the models perfectly is virtually impossible. This is when numerical experiments or simulation can be very useful. SciPy includes a large number of statistical distributions.

For discrete event simulation, SimPy is a very interesting and useful application. It's not yet integrated with SciPy/NumPy but, hopefully, it soon might be.

In this article, let's simulate queuing delays in a bank, building on top of the excellent tutorial included with the SimPy documentation (see the *References* section). The exercise can be extended by using statistical distributions from the SciPy package, along with its simulation framework.

Statistical distributions

You can explore the Erlang distribution [*http://en.wikipedia.org/wiki/Erlang_distribution*]—a good option for modelling the customers' arrival time and the time spent servicing each one. It is still widely used in the telecommunications industry. Try the following code:

```
import matplotlib.pyplot as plt
import scipy as sp
from scipy.stats import erlang
# 3 variations with mean = 2
er1=erlang(1,scale=2)
print 'mean, variance', er1.stats()
er2=erlang(2,scale=1)
print 'mean, variance', er2.stats()
er3=erlang(10,scale=.2)
print 'mean, variance', er3.stats()
# plot the probability distribution fn
x = sp.linspace(0,5)
p=plt.plot(x, er1.pdf(x))
p=plt.plot(x, er2.pdf(x))
p=plt.plot(x, er3.pdf(x))
plt.show()
```

The plot in Figure 1 shows what the probability distribution functions look like for the Erlang distribution with the parameters chosen. The function is like an exponential distribution when the shape parameter is 1, and seems to be a good option to understand the pace of customer arrivals. It looks a lot like a normal distribution for larger values of the shape parameter. However, the distribution with a shape parameter of 2 looks like what you may have often experienced—no one leaves quickly and some seem to be at the counter for a long time. All three distributions have the same mean, but their variances differ. The output of this program will be as shown below:

```
mean, variance (array(2.0), array(4.0))
mean, variance (array(2.0), array(2.0))
mean, variance (array(2.0), array(0.40))
```